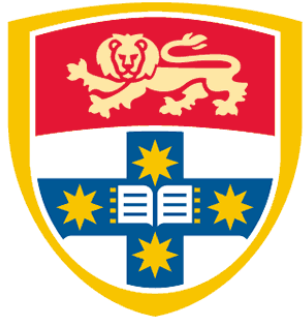




THE UNIVERSITY OF
SYDNEY

Project Report for ENGG2112

Bushfire Detection

Samiksha Aveg, 510574673, Software Engineering

Gayatri Motaparathi, 510671237, Software Engineering

Aidan Thomas, 520511398, Civil Engineering

FACULTY OF ENGINEERING

May 26, 2023

Executive Summary

Bushfires have consistently posed significant challenges in Australia due to the country's unique climatic conditions. The devastating impact of bushfires affects various stakeholders, including farmers, rural communities, flora and fauna, and emergency personnel. The emotional, economic, and ecological consequences are profound. This report aims to come up with a machine-learning system that helps in the detection of bushfires.

The objective of the project was to detect the bushfire activity with high accuracy rate from the data collected from the satellite images using the existing machine learning techniques. By training the program on data from satellite imagery and also integrating data from other satellites, in the future, like optical, infrared and radar imagery, and incorporating significant environmental parameters such as air temperature, relative humidity, barometric pressure, sound, light intensity, soil moisture, wind speed and direction, as well as gases including CO, CO₂, methane, H₂, and hydrocarbons, in addition to smoke detection, we aim to predict the future occurrences of the bushfire with high accuracy. This can help the government authorities and fire agencies to respond faster and allocate and mobilise resources faster to reduce the consequences of the bushfires.

We have sourced our data from Kaggle. The data was collected by NASA's satellites, MODIS (Moderate Resolution Imaging Spectroradiometer) and VIIRS (Visible Infrared Imaging Radiometer Suite) which observe and capture data about Earth's surface, atmosphere, and oceans. During the Data pre-processing, similar datasets were merged together and datasets were cleaned by removing the null values. The confidence rate was encoded for better prediction accuracy. Feature extraction and selection were performed to identify relevant attributes for training the machine learning models. The predictions were done using five supervised learning methods, which are, Linear Regression, KNN Classification, KNN Regression, Multi-Layer Perceptron (MLP), and Feedforward Neural Network. The performance of each model was evaluated using metrics such as R square score, Mean squared error, Accuracy score, ROC graph, F1 score, and Test loss.

Even though working with Linear regression was simple, it did not give satisfactory results, which indicates that the relationship between the input data and the target variable is not linear. KNN classification had a great performance, it had high prediction accuracy and weighted F1 score for both the datasets. KNN regression had a perfect Means squared error score but a lower accuracy rate compared to the other models, even though the value of 'k' was the same as the KNN classifier. Multi-Layer Perceptron had an excellent performance, with high prediction accuracy, slightly greater than KNN classification. Feedforward neural networks showed a low prediction accuracy rate with a high test loss value, which indicates that there has been an error in computation or reporting the loss value. The models with high prediction accuracy were picked as our best models, which are KNN classification and Mutli-Layer Perceptron. These models have a great ability to handle large datasets efficiently and also work well with the kind of data we are using to predict.

The practical implementation of our project in commercial settings holds the potential to revolutionise bushfire detection and monitoring. With the future bushfire prediction and live update system, we can keep the general public more aware of the bushfire situation around them. By incorporating advanced data processing techniques, integrating multiple detectors, and leveraging additional environmental parameters and weather data, we can significantly increase the confidence levels of fire detection, thus offering a valuable tool for mitigating the impact of bushfires in Australia.

CONTENTS

1. BACKGROUND AND MOTIVATION
2. OBJECTIVES AND PROBLEM STATEMENT
3. METHODOLOGY
 - 3.1 DATA PREPROCESSING
 - 3.2 FEATURE EXTRACTION
 - 3.3 METHODS
 - 3.4 SIMULATION ENVIRONMENT
4. SIMULATION RESULTS
 - 4.1 KEY FINDINGS AND SIGNIFICANCE
 - 4.2 ISSUES FACED
5. POTENTIAL FOR WIDER ADOPTION
6. CONCLUSIONS

BACKGROUND AND MOTIVATION

Bushfires have consistently posed a significant and recurring natural hazard in Australia, imparting profound challenges to diverse stakeholders. The distinctive climatic conditions prevailing within the country render it particularly susceptible to the ignition and rapid propagation of fires. On an annual basis, Australian firefighters attend to an average of 45,000 to 60,000 fires across the nation.

Bushfires have a far-reaching impact on a wide spectrum of stakeholders, including farmers, the rural populace, indigenous flora and fauna, and emergency personnel. The ferocity and rapidity exhibited by these fires engender extensive devastation, leaving behind a trail of destruction. Among the most distressing facets of bushfires are their tangible ramifications on individuals and communities. Historical records spanning the period from 1967 to 2013 show that fires engendered approximately 8,000 injuries and claimed the lives of 433 individuals. Astonishingly, the fatalities attributed to bushfires constituted roughly half of the total deaths arising from significant natural disasters during that time frame. The recent "Black Summer" bushfire season (2019-2020) incinerated over 19 million hectares of land.

Beyond the immediate physical consequences, bushfires exert profound emotional implications upon those affected. The traumatic experiences endured during large-scale conflagrations frequently inflict indelible wounds upon the mental well-being of individuals and communities. The anxiety and distress stemming from emergency evacuations, the loss of residences, cherished pets, personal possessions, and livelihoods, further compound the emotional toll. Moreover, certain communities encountered hindrances while attempting to evacuate expeditiously, owing to the unavailability of fuel stations amidst electricity outages or impassable thoroughfares, thereby leaving them ensnared in high-risk areas. In certain instances, resorting to seeking refuge upon beaches and watercraft. Such traumatic episodes wield enduring ramifications for mental health, imposing a substantial burden upon the impacted communities.

The economic ramifications of bushfires are pervasive and permeate various sectors of the Australian economy. The searing heat generated by these fires can engender the melting of edifices, culminating in the partial or complete annihilation of residential and commercial structures. Consequently, households and businesses suffer grievous financial losses. In February 2020 exclusively, early insurance claims reached an astonishing valuation of 1.9 billion Australian dollars, thereby underscoring the magnitude of economic devastation. The agricultural and tourism industries bear the brunt of these fires, with severe instances precipitating the cessation of business operations. The repercussions upon specific sectors, such as meat, wool, and honey production, prove particularly substantial, as regions profoundly impacted by bushfires witnessed a diminution of 13% in the national sheep flock. Correspondingly, areas partially affected sustained an additional decline of 17% within their sheep flock, according to data proffered by Meat & Livestock Australia.

The ecological consequences arising from bushfires are equally disconcerting, as they culminate in the eradication of invaluable native flora and fauna. The Black Summer bushfires alone resulted in an estimated three billion animal fatalities. Additionally, the decimation of habitats and food sources precipitated the demise of roughly one billion additional animals in the post-fire aftermath. The devastation inflicted upon forests, which harbour in excess of 80% of terrestrial species, engenders a grievous loss of biodiversity essential for the long-term sustenance of humanity and the ecosystem. Our data was sourced from Kaggle, but more specifically the data is recorded and compiled from NASA's satellite's MODIS and VIIRS which launched in 1999 and 2002 respectively. These satellites were fitted with the Advanced Spaceborne Thermal Emission and Reflection Radiometer (ASTER), the Clouds and Earth's Radiant Energy System (CERES), the Multi-angle Imaging SpectroRadiometer (MISR), the Measurement of Pollution in the Troposphere (MOPITT), and the Moderate Resolution Imaging Spectroradiometer. Overall, delivering stunning imagery of Earth and critical data about its

atmosphere, ocean, and terrestrial surface. The data has been used in assessments of aerosol content, weather prediction and, when taken together, they provide an invaluable record of the Earth system and how it has changed over time.

OBJECTIVES AND PROBLEM STATEMENT

Presently, firefighters receive their initial bushfire notification within a time span of 15 minutes from the moment of ignition. Consequently, this delay enables bushfires to expand prior to an arrival of emergency responders. Our project endeavours to mitigate the notification period by using satellite imagery and environmental data to identify and observe bushfire activity. Our primary objective is to develop and train a machine learning program that can accurately discern the ignition of bushfires with a notably high level of confidence using previous satellite imagery data.

Once trained, the program can be adopted on a wider spectrum, allowing for new satellite imagery which updates at a constant frequency to be discerned within the machine. Fire ignition can then be identified, pinpointed via longitude and latitude and alert required authorities. The machine will then provide ongoing notifications as the fire spreads to new locations.

Ultimately, the intended outcome of this initiative is to equip relevant authorities with comprehensive and reliable data, facilitating effective measures to combat bushfires and efficiently allocate resources, thereby mitigating the detrimental consequences brought about by such incidents.

METHODOLOGY

DATASET: The consists of 4 sub-datasets classified by satellite - MODIS and VIIRS - each having an Archive and NRT (Near Real Time) data which has records between sept 1, 2019 to Dec 31, 2019 and Jan 1, 2020 to Jan 31, 2020 respectively.

3.1 Data Pre-Processing

The first step was to understand the attributes given in each of the dataset and what it means. Then we merged the Archive and NRT datasets, leaving us with two datasets based on the satellite name - MODIS and VIIRS and these were named : M6 and V1 respectively. These two datasets have been maintained throughout the project as the radius of each hotspot detected by both satellites differ - the radius of fire pixel measured by MODIS is 1 km while VIIRS has improved spatial resolution of 375m.

After loading the datasets, the `info()` function was used to see the total number of columns and the number of non-null records it has along with its data type given in the form of a table. As we are using the 'confidence' attribute to help in the present prediction of fire, we have to ensure that this column is of numerical data type. Since, we know from the table that the 'confidence' column for V1 dataset is of 'object64' type, we used label encoding and created a new column named 'confidence_encoded' containing the numerical values for its counterparts. For example: the values of 'confidence' consisted of 'l' (for low confidence), 'n' (for nominal confidence), 'h' (for high confidence) were mapped to 0, 1, 2 respectively and the 'confidence_encoded' contains those numerical values.

Similarly, even though the values of 'confidence' column in dataset M6 were of the data type 'int64', it was divided into 4 quartiles and the values falling under the first and second quartiles were mapped to 0, values falling in the third quartile were mapped to 1 and values falling in the fourth quartile were

mapped to 2. This is done to maintain uniformity throughout the project. Other attributes which weren't that important like 'instrument', 'version' etc. weren't changed and left as it is.

Then the rows containing null values were dropped from the dataset. Otherwise, it would have led to distorted results due to inaccurate predictions by the machine learning methods and providing unreliable and misleading insights to the data leading to flawed interpretations.

Additionally, geopandas and matplotlib's pyplot libraries were used to scatterplot the data points using latitude and longitude attributes. The 'confidence' values for M6 and 'confidence_encoded' values for V1 were used as the basis of the color. The higher the confidence value, the darker is the color of the plotted point. This was done to visualize the hotspots detected by the satellites for the said time frame and help in better understanding of the data.

3.2 Feature Extraction

Since, we don't have very large data consisting of more than 50 attributes, feature extraction wasn't required. Instead Feature Selection was performed on the basis of the correlation table made for both the datasets to see correlation between any two attributes of the data which will later also help us in choosing input features for the machine learning methods. Originally, the datasets had 15 features then 'confidence_encoded' attribute was added to both and after feature selection we'll be using 9 features out of those 16 features.

3.3 Methods

After completing the data pre-processing and feature selection we now have data ready for training and testing using several machine learning methods. We have used the following 5 supervised machine learning methods as a part of our analysis :

3.3.1 LINEAR REGRESSION

This model is popular for its simplicity and interpretability and is also appropriate for continuous target variable - 'confidence_encoded' attribute in our case. The issue with this particular model is that it assumes linear relationship between the input features and the target variables i.e. $\mathbf{y} = \mathbf{X}\mathbf{\beta} + \epsilon$ where $\mathbf{y}$: target variable ; $\mathbf{X}$: data ; $\mathbf{\beta}$: coefficients ; ϵ : observation noise. To counter this problem, KNN Classification model was then used to confirm if the data shows non-linear or linear patterns.

3.3.2 KNN CLASSIFICATION

This proximity-based model is able to handle non-linear relationships of data quite well and makes no assumption about underlying data distribution. Furthermore, it doesn't require extensive parameter training or complex model training. But faced an issue while choosing the value of 'k' as if it is too small then the model becomes under fitted and if it is too large then it becomes overfitted. Hence, we used different values of 'k' to the best value where both datasets were giving optimum results.

3.3.3 KNN REGRESSION

This model adapts well to different datasets and problem domains along with ability to capture non-linear relationship between input feature and target variable. The value of k being used is the same as the one we used in the KNN Classifier. The weighted KNN Regression has been used to give more weight points to the neighbor nearest to the query point as it predicts the target by using local interpolation of nearest neighbors and this will enhance the model to predict the target with better precision and output better results.

3.3.4 MULTI-LAYER PERCEPTRON (MLP)

This type of artificial neural network uses multiple hidden layers and any number of neurons per hidden layer to understand the non-linear relationship between the input features and target variable and is also effective in learning and representing patterns within the satellite's imagery data. For our model, we ran several tests having different values of hidden layers to come upon the certain number where the model gave optimal results with best accuracy.

3.3.5 FEEDFORWARD NEURAL NETWORK

This model can automatically learn hierarchical representation of data with multiple layers. It is able to capture intricate non-linear relationships and learn complex relationships in data. For this model, repeated testing was done using various values of hidden layers until we came upon a certain number where the neural network presented best results.

3.4 Simulation Environment

The data has been split into an 80:20 ratio where 80% of the data is used for training and the remaining 20% for testing using the 'train_test_split' function. The simulation has majorly been performed using Google Colab through Python notebooks. The best model was chosen on the basis of the results given by each method. The performance measures varies from model to model but mainly the following metrics have been used to evaluate the model:

Performance metrics	Models used in
r2_score	Linear Regression KNN Regression
mean_squared_error	Linear Regression KNN Regression
accuracy_score	KNN Classification Multi-Layer Perceptron Feedforward Neural Network
f1_score	KNN Classification
test_loss	Feedforward Neural Network

SIMULATION RESULTS

The Simulation Results for each model has been explained below under their names:

4.1 Key Findings and Significance

The Key Findings of each model has been explained below under their names:

4.1.1 LINEAR REGRESSION

It has been ensured that the data split for training and testing remains same every time the code is executed by using 'random_state' parameter, hence allowing reproducibility of results. The following performance table shows the result obtained for both the datasets:

Performance metrics	V1 dataset	M6 dataset
r2_score	0.13	0.45
mean_squared_error	0.16	0.38

The r2_score for M6 and V1 datasets are 0.45 and 0.13 respectively. This suggests that there is low level of correlation between the input features and the target variable for linear relationship and there is a high value of variance (refers to the measure of difference of the observed values differ and the average of predicted values, ideally its value should be low for high accuracy) between them.

The mean_squared_error for M6 and V1 datasets are 0.38 and 0.16 respectively. This shows that the model makes quite a lot of errors in prediction of the target. Ideally, mean_squared_error should be low for higher efficiency in making predictions.

The poor performance result insists that the data exhibits complex non-linear patterns or relations and not a linear relationship between the input features and the target variable as assumed by this model. Therefore, it is not a good fit for the datasets, thus, not working out.

4.1.2 KNN CLASSIFICATION

It has been ensured that the data for training and testing is split into consistent manner every time the code is executed so as to allow reproducibility of results.

The performance metrics used are: accuracy_score and f1_score and the following are the values for both the datasets:

Performance metrics	V1 dataset	M6 dataset
accuracy_score	0.958	0.852
weighted f1_score	0.95	0.85

The above table shows very good performance by the KNNClassifier for both the datasets. accuracy_score shows that the model is able to correctly predict the class labels for 85% of the instances belonging to M6 and 95% for V1 dataset. The weighted f1_score is the average of class labels weighted by the number of instances in each class so as to provide balanced evaluation. It indicates that the classification model was able to attain a good balance between precision and recall while minimising false positives and false negatives.

4.1.3 KNN REGRESSION

It has been ensured that the splitting of data for training and testing is split into remains the same every time the code is executed so as to allow the reproducibility of results. The following table shows the value of the metrics used to evaluate the performance of the model:

Performance metrics	V1 dataset	M6 dataset
r2_score	0.714	0.796
mean_squared_error	0.0	0.0

The r2_score shows that the model can predict the target for 71.4% of V1 dataset and 79.6% of M6 dataset accurately. The mean_squared_error being 0 means that there are no errors in the observed and predicted values. Hence, KNN Regression model is a reasonably good fit with room left for improvement.

4.1.4 MULTI-LAYER PERCEPTRON

It has been ensured that the data is split into consistent manner every time the code is executed so as to allow reproducibility of results. The following shows the value of performance metrics for both the datasets:

Performance metrics	V1 dataset	M6 dataset
accuracy_score	0.963	0.877
roc_auc	0.993 [Appendix 2]	0.968 [Appendix 1]

The table exhibits that the model has been really successful in making accurate predictions for the majority of the datasets. accuracy_score shows that the model has been able to predict the class label accurately for 96.27% of the V1 dataset and 87.72% of the M6 dataset. roc_auc being 0.968 for M6 dataset and 0.993 for V1 dataset, suggests that the model is exceptionally good at distinguishing between classes.

4.1.5 FEEDFORWARD NEURAL NETWORK

The train-test split is same by assigning an integer value to random_state to ensure data is split into consistent manner every time the code is executed, hence allowing reproducibility of results. The following performance table shows are the values for both the datasets:

Performance metrics	V1 dataset	M6 dataset
test accuracy	0.789	0.568
test loss	-1535646464.0	-9235617.0

The test loss is the summation of errors made for each interaction or epoch.

Its value for V1 and M6 dataset is -1535646464.0 and -9235617.0, correspondingly. Lower the loss, the better a model is for that particular data. Since, test loss is bigger for M6 dataset, hence the it is not a good fit for the data which is also reflected by the value of test accuracy for the dataset which is only 0.568 or 56.8%. While for V1 dataset, the accuracy is quite high in comparison, that is 0.789 or 78.9%. Therefore, showing that it's quite a good fit for V1 dataset in comparison.

KNN classification and Multi-Layer Perceptron have been chosen as the Final Models based on their performance compared to the rest of the models. Furthermore, while KNN classification is known for its simplicity and interpretability, Multi-Layer Perceptron is flexible and therefore, performs well on different datasets to find intricate patterns underlying in the dataset. These are few of the reasons along with the highest accuracy of the results displayed for both the datasets which led us to consider and ultimately, choose them as our best models.

4.2 Issues Faced

Making the model work while maintaining a high accuracy rate was not an easy task. The following are some of the challenges we encountered while doing the analysis for this project:

a) The accuracy for the M6 dataset wasn't as high as expected compared to the V1 dataset for all approaches. The highest accuracy found for the M6 dataset is 0.87 as opposed to 0.96 for the V1 dataset.

There has been a difference in the accuracy rate of the M6 dataset and the V1 dataset, where the M6 dataset has a lower accuracy rate when compared to the accuracy rate of the V1 dataset. After some research, we have found that the radius of the pixel can affect the accuracy rate. The radius of the pixel for the M6 dataset is 1000m while the radius of the pixel for the V1 dataset is 375m. If the size of the fire is smaller than the pixel size it results in low accuracy as the precise location of the individual fires is difficult to identify.

b) Using the attributes 'latitude' and 'longitude' was not initially a part of our analysis. When we tried to later use it in all the models, we saw that the KNN Classification and Regression had a significant increase in their F1 score and the mean squared error reduced to zero, while the other models showed a decrease in their accuracy rates.

We came to the conclusion that this might be due to the fact that the KNN algorithm takes the nearest neighbours into consideration when dealing with the similarities of data points. By adding spatial information like latitude and longitude the KNN model can more effectively predict while capturing the relationship between the data points.

c) The running time of Multi-Layer Perceptron and Feedforward Neural Network on Google Colab was another obstacle that we faced. This slowed down our process of experimenting with the number of hidden layers and the number of neurons used per hidden layer. Then we started working on VSCode, the model did run much faster compared to Google Colab. This gave us a chance to experiment with hidden layers and random states to achieve high accuracy.

POTENTIAL FOR WIDER ADOPTION

As our project on bushfire detection in Australia using machine learning methods progresses, we have some ideas for future development in mind.

Improvement One: Expanding Data Sources

To leverage the strengths of different satellite data sources, we will incorporate data from multiple sensors, including optical, infrared, and radar imagery. Similarly, the inclusion of historical weather data and predictions obtained from reputable sources such as the Bureau of Meteorology would facilitate the prediction of fire occurrences across the entire nation. Additionally, incorporating significant environmental parameters such as air temperature, relative humidity, barometric pressure, sound, light intensity, soil moisture, wind speed and direction, as well as gases including CO, CO₂, methane, H₂, and hydrocarbons, in addition to smoke detection, would contribute to the precision and reliability of fire detection. By leveraging this comprehensive dataset encompassing both environmental conditions and weather patterns, our project can enhance the accuracy and effectiveness of fire ignition predictions.

Improvement Two: Future Predictions

While our current model focuses on detecting existing fires, we recognize the importance of predicting fire incidents in advance. By leveraging machine learning algorithms, we will train our system to forecast the likelihood of future fire outbreaks based on the existing data along with the new data we plan to add as mentioned in Improvement One. This predictive capability will empower authorities and communities to take proactive measures, allocate resources effectively, and minimize the devastating impact of bushfires.

Improvement Three: Automation and Updates

We would like to explore methods to automate the integration of our satellite-based bushfire detection system with existing emergency response systems. The bushfire detection software could be incorporated into firefighter alert mechanisms and online alerts, giving residents updates on the movements and likelihood of fire spread, and enabling swift and coordinated actions.

Ultimately, commercialisation of our project would equip areas with a live and more informed detail of bushfire activity. By incorporating a variety of monitors, satellites and historical and predicted weather data our machine learning system would generate a detection to an accurate confidence level. Thereby, equipping areas with informed knowledge to better distribute resources and reduce response time which in turn limit the potential damages and casualties.

CONCLUSION

This report highlights the significant impact of bushfires in Australia. It emphasises the need for effective measures to combat bushfires and mitigate their detrimental consequences. We have worked towards our objective of the project which is to develop a machine learning program using data from satellite imagery to identify and observe bushfire activity, reducing the time taken for prediction to enable timely response.

The methodology section outlines the data pre-processing steps, including merging and cleaning the datasets obtained from NASA's satellites. Feature extraction and selection techniques were applied to prepare the data for training and testing using supervised machine learning methods. The report specifically focuses on five methods, which are, Linear Regression, KNN Classification, KNN Regression, Multi-Layer Perceptron (MLP), and Feedforward Neural Network.

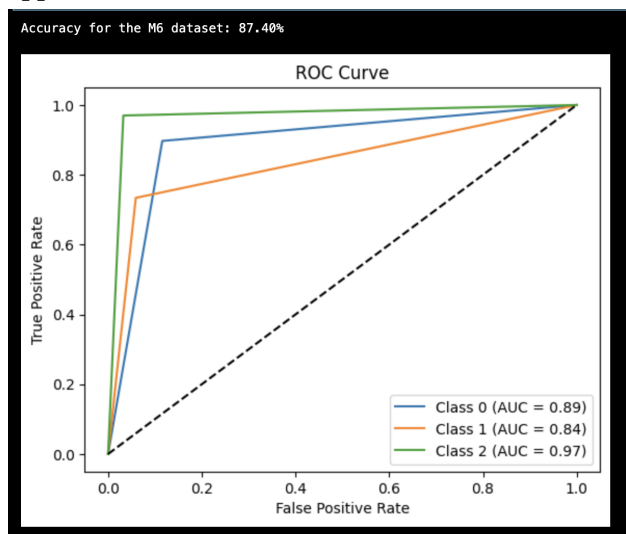
Among these methods, KNN Classification and MLP emerged as the final models selected for their high accuracy in predicting bushfire occurrences. The models have achieved an average accuracy score of 96% for the V1 dataset and 86% for the M6 dataset, indicating their ability to correctly predict confidence of occurrence based on the input data.

The economic and ecological significance of the project lies in equipping relevant authorities with comprehensive and reliable data, enabling effective resource allocation and combatting bushfires more efficiently. By reducing the prediction time and providing ongoing notifications as the fire spreads, the program can help minimise the devastating consequences of bushfires.

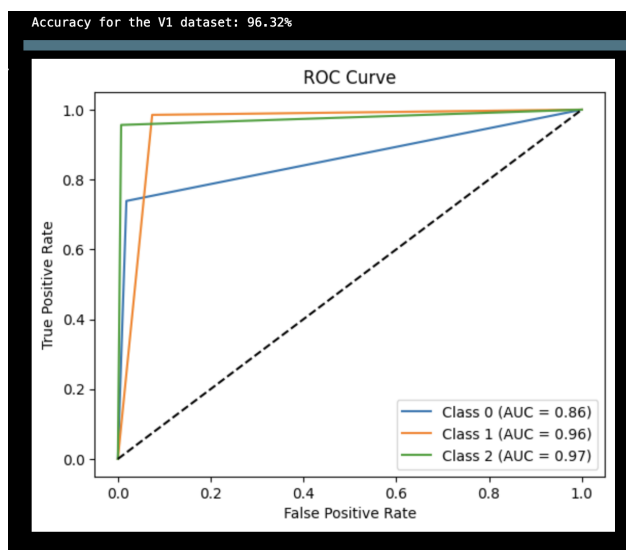
In conclusion, the project's use of machine learning, particularly KNN Classification and MLP models, shows great promise in predicting bushfire occurrences in Australia. By leveraging satellite imagery and integrating environmental data, these models can improve early detection and response to bushfire occurrences, thereby mitigating the physical, emotional, economic, and ecological impacts caused by these natural disasters.

APPENDIX

Appendix 1



Appendix 2



REFERENCES

DATASET:

bhat, N. (2020) Australian bush fire satellite data (NASA), Kaggle. Available at: https://www.kaggle.com/datasets/nagarajbhat/australian-bush-fire-satellite-data-nasa?select=fire_archive_M6_101673.csv (Accessed: 25 March 2023).

[1] How to interpret loss and accuracy for a machine learning model (1962) Stack Overflow. Available at: <https://stackoverflow.com/questions/34518656/how-to-interpret-loss-and-accuracy-for-a-machine-learning-model> (Accessed: 26 May 2023).

[2] Coefficient of determination (2023) Wikipedia. Available at: https://en.wikipedia.org/wiki/Coefficient_of_determination (Accessed: 26 May 2023).

[3] Mean square error & R2 score clearly explained (2018) BMC Blogs. Available at: <https://www.bmc.com/blogs/mean-squared-error-r2-and-variance-in-regression-analysis/> (Accessed: 26 May 2023).

[4] Sklearn.neural_network.MLPClassifier (no date) scikit. Available at: https://scikit-learn.org/stable/modules/generated/sklearn.neural_network.MLPClassifier.html#sklearn.neural_network.MLPClassifier (Accessed: 26 May 2023).

[5] Sklearn.neural_network.MLPClassifier (no date a) scikit. Available at: https://scikit-learn.org/stable/modules/generated/sklearn.neural_network.MLPClassifier.html#sklearn.neural_network.MLPClassifier (Accessed: 26 May 2023).

[6] Sklearn.neural_network.MLPClassifier (no date) scikit. Available at: https://scikit-learn.org/stable/modules/generated/sklearn.neural_network.MLPClassifier.html#sklearn.neural_network.MLPClassifier (Accessed: 26 May 2023).

[7] 1.6. nearest neighbors (no date) scikit. Available at: <https://scikit-learn.org/stable/modules/neighbors.html> (Accessed: 26 May 2023).

[8] Sklearn.neighbors.kneighborsregressor (no date) scikit. Available at: <https://scikit-learn.org/stable/modules/generated/sklearn.neighbors.KNeighborsRegressor.html#sklearn.neighbors.KNeighborsRegressor> (Accessed: 26 May 2023).

[9] Sklearn.neighbors.kneighborsclassifier (no date) scikit. Available at: <https://scikit-learn.org/stable/modules/generated/sklearn.neighbors.KNeighborsClassifier.html#sklearn.neighbors.KNeighborsClassifier> (Accessed: 26 May 2023).

[10] Supervised learning: Predicting an output variable from high-dimensional observations (no date) scikit. Available at: https://scikit-learn.org/stable/tutorial/statistical_inference/supervised_learning.html#linear-regression (Accessed: 26 May 2023). **[11] Sklearn.neural_network.MLPClassifier (no date a) scikit.** Available at: https://scikit-learn.org/stable/modules/generated/sklearn.neural_network.MLPClassifier.html#sklearn.neural_network.MLPClassifier (Accessed: 26 May 2023).

[13] Bureau of Meteorology, “Bushfire weather,” Bom.gov.au, 2017. Available at: <http://www.bom.gov.au/weather-services/fire-weather-centre/bushfire-weather/index.shtml> (Accessed: 26 May 2023)

- Overall, the addition of latitude and longitude attributes to the input features helps the model incorporate spatial information, contextual factors, and potential correlations, leading to improved accuracy in the classification task.



The F1 score is a commonly used metric for evaluating the performance of a binary classification model. It combines precision and recall into a single value and provides a balanced measure of a model's accuracy.



The F1 score is calculated as follows:

$$F1 = 2 * (\text{precision} * \text{recall}) / (\text{precision} + \text{recall})$$

However, in certain scenarios, the classes in the dataset might be imbalanced, meaning that there are significantly more instances of one class than the other. In such cases, evaluating the model's performance solely based on the F1 score may not be sufficient. This is where the weighted F1 score comes into play.

The weighted F1 score takes into account the class distribution and calculates a weighted average of the F1 scores for each class. The weight assigned to each class is proportional to the number of instances belonging to that class. It ensures that the F1 score of the minority class contributes meaningfully to the overall score.

The formula for calculating the weighted F1 score depends on the implementation or library being used. In scikit-learn, for example, the weighted F1 score is calculated using the `average` parameter. When set to 'weighted', it computes the weighted average F1 score.

$$\text{Weighted F1 score} = (n_1 * F1_1 + n_2 * F1_2 + \dots + n_k * F1_k) / (n_1 + n_2 + \dots + n_k)$$

where $F1_i$ is the F1 score of class i , and n_i is the number of instances belonging to class i .

In summary, the main difference between the F1 score and the weighted F1 score lies in how they handle class imbalance. The F1 score treats both classes equally, while the weighted F1 score takes into consideration the class distribution and assigns appropriate weights to each class. Therefore, the weighted F1 score is a more suitable metric when dealing with imbalanced datasets.



A feedforward neural network, also known as a multilayer perceptron (MLP), is a type of artificial neural network where information flows in only one direction, from the input layer through the hidden layers to the output layer. It is the simplest and most common type of neural network architecture.



The network is composed of multiple layers of interconnected nodes, called neurons, organized into three main types of layers: the input layer, one or more hidden layers, and the output layer. Each neuron in a layer is connected to every neuron in the next layer, but there are no connections between neurons within the same layer or between neurons in previous layers.

Here's a brief overview of the feedforward neural network process:

1. **Input Layer:** The input layer consists of neurons that receive input data, which can be numerical values or encoded features. Each neuron represents a specific feature or attribute of the input data.
2. **Hidden Layers:** The hidden layers are intermediate layers between the input and output layers. They perform complex computations by applying weights and biases to the input from the previous layer. Each neuron in the hidden layer takes inputs from all the neurons in the previous layer and passes the output to the next layer.
3. **Activation Function:** Each neuron in the hidden and output layers typically applies an activation function to introduce non-linearity into the network. Common activation functions include the sigmoid, tanh, and rectified linear unit (ReLU) functions. The activation function determines the output of the neuron based on its input.
4. **Output Layer:** The output layer produces the final output of the network. It usually represents the predicted class labels or continuous values, depending on the type of problem being addressed. The number of neurons in the output layer corresponds to the number of possible outputs.
5. **Forward Propagation:** The input data flows through the network in a forward direction. Each neuron calculates a weighted sum of its inputs, applies the activation function, and passes the result to the next layer.
6. **Loss Function:** The output of the network is compared to the expected output using a loss function, which quantifies the difference between the predicted and target values. The choice of loss function depends on the nature of the problem, such as mean squared error (MSE) for regression or cross-entropy for classification.
7. **Training:** During the training phase, the network adjusts its weights and biases to minimize the loss function. This is typically done using optimization algorithms like gradient descent, which iteratively updates the weights in the direction that reduces the loss.
8. **Backpropagation:** Backpropagation is the process of propagating the error backward through the network. It calculates the gradient of the loss function with respect to each weight in the network, allowing the weights to be updated accordingly.
9. **Repeat:** Steps 5 to 8 are repeated for multiple iterations or epochs until the network converges and achieves satisfactory performance on the training data.

Feedforward neural networks can be used for a variety of tasks, including pattern recognition, classification, regression, and function approximation. They are powerful models for capturing complex relationships in data and have been successfully applied in various domains, such as computer vision, natural language processing, and financial forecasting.



Feedforward neural networks, also known as multilayer perceptrons (MLPs), have several advantages and limitations. Let's explore them:



Advantages of Feedforward Neural Networks:

1. **Universal Approximators:** Feedforward neural networks are universal function approximators, meaning they can approximate any arbitrary function given sufficient complexity in the network architecture. This flexibility allows them to model and learn complex patterns and relationships in data.
2. **Nonlinear Mapping:** Feedforward neural networks can learn and represent nonlinear mappings between input and output variables. They can capture intricate nonlinear relationships that might be challenging for traditional linear models.
3. **Ability to Learn from Large Datasets:** Feedforward neural networks can benefit from large datasets. With a large amount of data, they can learn robust representations and generalize well to unseen examples.
4. **Feature Learning:** Deep feedforward neural networks with multiple layers can automatically learn hierarchical representations of data. Each layer captures increasingly abstract and high-level features, enabling the network to learn complex representations.
5. **Parallel Processing:** Feedforward neural networks can take advantage of parallel processing on modern hardware, such as GPUs (Graphics Processing Units), which can significantly speed up training and inference.

Limitations of Feedforward Neural Networks:

1. **Lack of Interpretability:** Neural networks, including feedforward networks, are often considered black-box models. They provide accurate predictions but offer limited interpretability. Understanding how the network arrives at its predictions can be challenging.
2. **Overfitting:** Feedforward neural networks are prone to overfitting, especially when dealing with small datasets or when the model capacity is too high relative to the complexity of the problem. Overfitting occurs when the network learns to perform well on the training data but fails to generalize to new, unseen examples.
3. **Computational Complexity:** Training deep feedforward neural networks can be computationally expensive, especially when dealing with large datasets and complex network architectures. Training times can be lengthy, and resource requirements can be high, particularly for deep networks.
4. **Data Requirements:** Feedforward neural networks often require a substantial amount of labeled training data to learn effectively. Insufficient data may lead to poor generalization and performance.
5. **Hyperparameter Sensitivity:** Feedforward neural networks have several hyperparameters, such as the number of layers, number of units per layer, learning rate, and regularization parameters. Tuning these hyperparameters can be challenging, and improper settings may lead to suboptimal performance.

Understanding the advantages and limitations of feedforward neural networks is important when choosing and designing models for specific tasks. It's crucial to consider the characteristics of the dataset, interpretability requirements, available computational resources, and other factors to determine whether a feedforward neural network is suitable for a particular problem.